



KLEINES PRIVATES LEHRINSTITUT

DERKSEN

G Y M N A S I U M

Informatik 09 - Objektorientierte Programmierung mit Java

Programmieren: Sprachen

Karol-Java Übersetzung

Übersicht Befehlsarten

Semikolon und geschweifte Klammern

Strukturierung von Programmcode - Syntax

Strukturierung von Programmcode - Semantik:
Methoden

Übersetzungstraining: Methoden

Karol mit Java und Struktogramme

Programmablauf grafisch darstellen: Struktogramme

Struktogramm-Training

Taschenrechner

Methodenköpfe

Methoden im Klassendiagramm

Attribute im Klassendiagramm

Attribute im Klassendiagramm

Struktogramme und Klassendiagramme

Übersicht: Variablen

Besondere Methoden: Getter und Setter

Besondere Methoden: Konstruktoren

Besondere Methoden: Main/Hauptprogramm

Speichern

Blöcke

Code

// Kommentar

Beenden

MarkeSetzen

MarkeLöschen

LinksDrehen 1

RechtsDrehen 1

Schritt 1

Hinlegen 1

Aufheben 1

wiederhole 10 mal

wiederhole solange

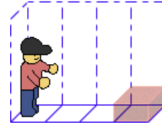
Start

Start

Herzlich Willkommen! Hier lernst du Schritt für Schritt die Welt von Karol kennen. Das Tutorial zeigt dir die ersten Grundlagen für die Programmierung.

Tutorial anzeigen

Lege einen Ziegel



← zurück zur Übersicht

Struktogramm

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom Maschinensprache übersetzt werden. in

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung** ). Dem gegenüber steht die **Logik eines Programms (=**) die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol**  übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie** **oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen

bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).  Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten Sprache Java**.

Programmieren: Sprachen



Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom **Compiler** in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= Syntax)**. Dem gegenüber steht die **Logik eines Programms (= Semantik)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie Python oder JavaScript** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).

Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen



Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom **Compiler** in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= Syntax)**. Dem gegenüber steht die **Logik eines Programms (=)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).

Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen



Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom **Compiler** in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= Syntax)**. Dem gegenüber steht die **Logik eines Programms (= Semantik)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie** **oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).

Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen



Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom **Compiler** in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= Syntax)**. Dem gegenüber steht die **Logik eines Programms (= Semantik)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie Java oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).

Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen



Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom **Compiler** in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= Syntax)**. Dem gegenüber steht die **Logik eines Programms (= Semantik)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie Java oder Python** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).

Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.



1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

MarkeSetzen

Schritt 10

wiederhole immer

LinksDrehen 1

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

Schritt 10

wiederhole immer

LinksDrehen 1

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

```
karol.markeSetzen();
```

Schritt 10

wiederhole immer

LinksDrehen 1

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

```
karol.markeSetzen();
```

Schritt 10

```
karol.schritt(10);
```

wiederhole immer

LinksDrehen 1

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

```
karol.markeSetzen();
```

Schritt 10

```
karol.schritt(10);
```

wiederhole immer

LinksDrehen 1

```
while(true) {  
    karol.linksDrehen(1);  
}
```

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

```
karol.markeSetzen();
```

Schritt 10

```
karol.schritt(10);
```

wiederhole immer

LinksDrehen 1

```
while(true) {  
    karol.linksDrehen(1);  
}
```

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

```
karol.markeSetzen();
```

Schritt 10

```
karol.schritt(10);
```

wiederhole immer

LinksDrehen 1

```
while(true) {  
    karol.linksDrehen(1);  
}
```

wiederhole 10 mal





1. Bearbeite die Aufgaben von Robot Karol online: karol.arrrg.de
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

```
karol.schritt(2);
```

MarkeSetzen

```
karol.markeSetzen();
```

Schritt 10

```
karol.schritt(10);
```

wiederhole immer

LinksDrehen 1

```
while(true) {  
    karol.linksDrehen(1);  
}
```

wiederhole 10 mal



Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten



Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Semikolon und geschweifte Klammern



Wann macht man in Java ein Semikolon und wann setzt man geschweifte Klammern ein?

1. Überlege zuerst allein (2 Minuten)
2. Besprecht eure Überlegungen zu zweit und probiert sie in Online-Karol zur bestätigen (4 Minuten)

Strukturierung von Programmcode - Syntax



In Java wird Programmcode vor allem mit

strukturiert:

markieren das Ende eines **einfachen**

Befehls oder Methodenaufrufs (normalerweise am Ende der Zeile).

markieren **Code-Abschnitte**,

die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.



Computer Science Student :



Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **Semikolons und geschweiften Klammern** strukturiert:

markieren das Ende eines **einfachen**

Befehls oder Methodenaufrufs (normalerweise am Ende der Zeile).

markieren **Code-Abschnitte**,

die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.



Computer Science Student :



Strukturierung von Programmcode - Syntax



In Java wird Programmcode vor allem mit **Semikolons und geschweiften Klammern** strukturiert:

- **Semikolons** markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).

markieren **Code-Abschnitte**,

die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.



Computer Science Student :



Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **Semikolons und geschweiften Klammern** strukturiert:

- **Semikolons** markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- **Geschweifte Klammern** markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.



Computer Science Student :



Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **Semikolons und geschweiften Klammern** strukturiert:

- **Semikolons** markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- **Geschweifte Klammern** markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu **innerhalb** der Klammern auch von der Klasse, Methode, Schleife, etc.



Computer Science Student :



Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **Semikolons und geschweiften Klammern** strukturiert:

- **Semikolons** markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- **Geschweifte Klammern** markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu **innerhalb** der Klammern auch von **innerhalb** der Klasse, Methode, Schleife, etc.



Computer Science Student :



Strukturierung von Programmcode - Semantik: Methoden



Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem _____, welche Eigenschaften die Methode hat und anschließend im _____, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=aufgerufen) werden.

Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass reduziert wird, der Code durch aussagekräftige _____ besser verstanden werden kann und Fehler schneller gefunden werden.

In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.

Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {  
  
    karol.schritt(2);  
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden



Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem **Methodenkopf**, welche Eigenschaften die Methode hat und anschließend im **Methodenrumpf**, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=aufgerufen) werden.

Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass reduziert wird, der Code durch aussagekräftige **Methoden** besser verstanden werden kann und Fehler schneller gefunden werden.

In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.

Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {  
  
    karol.schritt(2);  
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden



Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem **Methodenkopf**, welche Eigenschaften die Methode hat und anschließend im **Methodenrumpf**, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=aufgerufen) werden.

Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass reduziert wird, der Code durch aussagekräftige besser verstanden werden kann und Fehler schneller gefunden werden.

In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.

Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {  
  
    karol.schritt(2);  
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden



Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem **Methodenkopf**, welche Eigenschaften die Methode hat und anschließend im **Methodenrumpf**, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=aufgerufen) werden.

Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass **doppelter Code** reduziert wird, der Code durch aussagekräftige besser verstanden werden kann und Fehler schneller gefunden werden.

In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.

Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {  
  
    karol.schritt(2);  
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden



Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem **Methodenkopf**, welche Eigenschaften die Methode hat und anschließend im **Methodenrumpf**, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=aufgerufen) werden.

Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass **doppelter Code** reduziert wird, der Code durch aussagekräftige **Methodennamen** besser verstanden werden kann und Fehler schneller gefunden werden.

In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.

Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

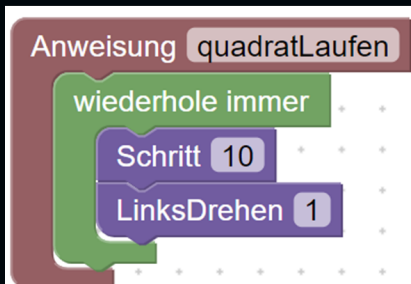
```
void tueEtwas() {  
  
    karol.schritt(2);  
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

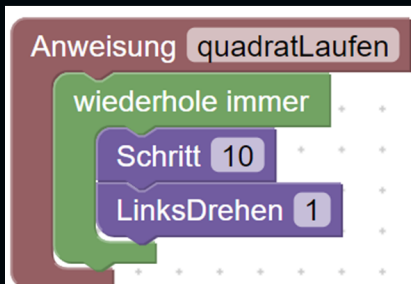


Übersetze die folgenden Code-Schnippsel (**code snippets**) in Java-Code ohne den Computer zu verwenden.





Übersetze die folgenden Code-Schnippsel (**code snippets**) in Java-Code ohne den Computer zu verwenden.

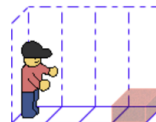




- Bearbeite die Robot Karol (karol.arrrg.de) Level erneut. Schreibe den Programmcode dieses Mal direkt in Java!
Zur Erinnerung: Deinen Code schreibst du an der gelb markierten Stelle in der main-Methode.
- Zeige nach den Schritten jeweils das Struktogramm an und notiere auf einem [Schmierzettel](#), wie du die letzte Spalte in Hefteintrag 8 befüllen würdest.
- Was passiert, wenn du dein Programm aus eigenen Methoden aufbaust? Wieso ist das so?

```
1 class Programm {  
2     Robot karol = new Robot();  
3  
4     void main() {  
5                               
6     }  
7 }
```

Lege einen Ziegel



← zurück zur Übersicht

Struktogramm

Programmablauf grafisch darstellen: Struktogramme



Strukturgramme (auch: **Struktogramme**) stellen **Algorithmen (=Ablauf eines Programms)** grafisch dar. Ein Struktogramm bezieht sich immer auf genau eine Methode und stellt aufgerufene Methoden als ein Element dar.

In **Hefteintrag 8** sind die Struktogramm-Bausteine für verschiedene Befehlsarten dargestellt, die beliebig miteinander kombiniert werden können.

Zusätzlich gibt es folgenden Baustein, der eine leere Stelle (z.B. leerer else-Block) darstellt:

Programmablauf grafisch darstellen: Struktogramme



Strukturgramme (auch: **Nassi-Shneiderman-Diagramme**) stellen **Algorithmen (=Ablauf eines Programms)** grafisch dar. Ein Struktogramm bezieht sich immer auf genau eine Methode und stellt aufgerufene Methoden als ein Element dar.

In **Hefteintrag 8** sind die Struktogramm-Bausteine für verschiedene Befehlsarten dargestellt, die beliebig miteinander kombiniert werden können.

Zusätzlich gibt es folgenden Baustein, der eine leere Stelle (z.B. leerer else-Block) darstellt:

Zeichne zu den folgenden Code Snippets jeweils ein Struktogramm:



```
ziegelImNordenLegen();  
quadratLaufen();  
tanzen();
```

```
void quadratLaufen() {  
    while (true) {  
        karol.schritt(10);  
        karol.linksDrehen();  
    }  
}
```



Zeichne zu den folgenden Code Snippets jeweils ein Struktogramm:



```
ziegelImNordenLegen();  
quadratLaufen();  
tanzen();
```

```
void quadratLaufen() {  
    while (true) {  
        karol.schritt(10);  
        karol.linksDrehen();  
    }  
}
```



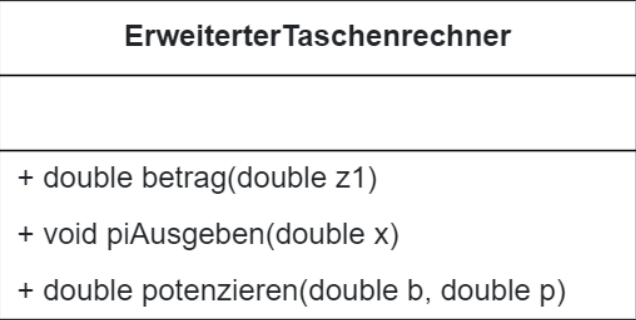
Implementiere das Klassendiagramm rechts mit Hilfe von Blockly. Probiere deine Methode aus (siehe Anleitung auf der Vorderseite). Fülle die Lücken im Java-Code unten aus, wenn deine Methoden funktionieren.

```
public class Taschenrechner
{
    public
```

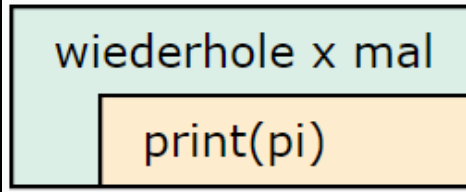
Taschenrechner
+ double addieren(double z1, double z2)
+ double subtrahieren(double z1, double z2)
+ double multiplizieren(double z1, double z2)
+ double dividieren(double z1, double z2)
+ double mittelwert(double z1, double z2)

Setze folgende Diagramme in BlueJ-Blockly um und notiere den Programmcode auf der nächsten Seite

Klassendiagramm:



Struktogramm für piAusgeben(double x):



Zeichne hier die Struktogramme für die restlichen Methoden:

```
public class ErweiterterTaschenrechner
{
    public

}
public
```


Methodenköpfe



Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt:

Die verwendeten `Methodenköpfe` haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: `String`, Wahrheitswert: `boolean`)

Rückgabetyp - Datentyp des Rückgabewerts (= `int`). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }
```

```
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt:

Die verwendeten **Zugriffsmodifizierer** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: **int**, Kommazahlen: **double**, Text: **String**, Wahrheitswert: **boolean**)

Rückgabetyp - Datentyp des Rückgabewerts (= **Datentyp**). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyt name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Zugriffsmodifizierer** haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: `String`, Wahrheitswert: `boolean`)

Rückgabetyt - Datentyp des Rückgabewerts (= `int`). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyt - void bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Schlüsselwörter** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: `String`, Wahrheitswert: `boolean`)

Rückgabetyp - Datentyp des Rückgabewerts (= `int`). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabety name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Schlüsselwörter** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: **int** , Kommazahlen: , Text: , Wahrheitswert:)

Rückgabety - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabety - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabety name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Schlüsselwörter** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: **int** , Kommazahlen: **double** , Text: **String** , Wahrheitswert: **boolean**)

Rückgabety - Datentyp des Rückgabewerts (= **Datentyp**). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabety - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Schlüsselwörter** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann 'überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: **int** , Kommazahlen: **double** , Text: **String** , Wahrheitswert: **boolean**)

Rückgabetyp - Datentyp des Rückgabewerts (= **Datentyp**). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabety name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Schlüsselwörter** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann 'überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: **int** , Kommazahlen: **double** , Text: **String** , Wahrheitswert: **boolean**)

Rückgabety - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabety - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe



Im **Methodenkopf** können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }  
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: **Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name**

Die verwendeten **Schlüsselwörter** haben folgende Bedeutungen:

Zugriffsmodifizierer: **public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: **int** , Kommazahlen: **double** , Text: **String** , Wahrheitswert: **boolean**)

Rückgabetyp - Datentyp des Rückgabewerts (= **return-value**). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methoden im Klassendiagramm



Taschenrechner

- void einschalten()

+ double addieren(double z1, double z2)

```
public class Taschenrechner {  
    private void einschalten() {  
        // ...  
    }  
    public double addieren(double z1, double z2) {  
        return z1 + z2;  
    }  
}
```

Attribute im Klassendiagramm

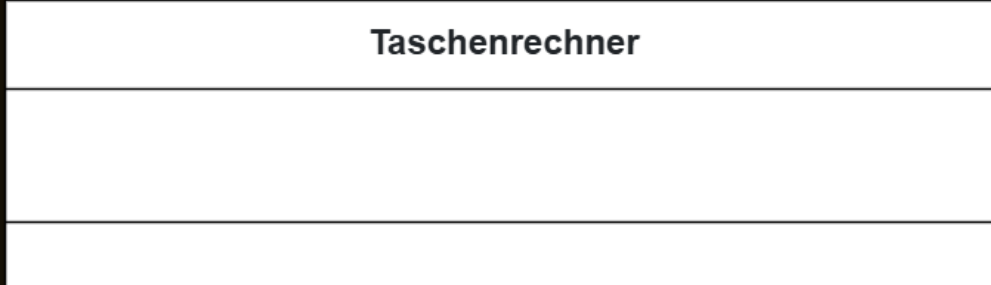


Findet heraus, wie Attribute im Klassendiagramm dargestellt werden. Achtet auf **Zugriffsmodifizierer, Datentyp und Name**. Macht eine Skizze auf einem Schmierblatt.

Attribute im Klassendiagramm



Attribute werden im Klassendiagramm folgendermaßen dargestellt:



Struktogramme und Klassendiagramme





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x
5), **x != 5** (x
5), **text.equals("Hallo")** (text
)





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x **ist gleich** 5), **x != 5** (x **ist nicht gleich** 5), **text.equals("Hallo")** (text **ist gleich** "Hallo")





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x **ist gleich** 5), **x != 5** (x **ungleich** 5), **text.equals("Hallo")** (text
,





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x **ist gleich** 5), **x != 5** (x **ungleich** 5), **text.equals("Hallo")** (text **ist "Hallo"**),





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x **ist gleich** 5), **x != 5** (x **ungleich** 5), **text.equals("Hallo")** (text **ist "Hallo"**),





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x **ist gleich** 5), **x != 5** (x **ungleich** 5), **text.equals("Hallo")** (text **ist "Hallo"**),





Wertzuweisung mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

Verwendung des Werts mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

Vergleich des Werts mit **<, <=, ==, >=, >, !=, name.equals(Wert)**

Beispiele: **x == 5** (x **ist gleich** 5), **x != 5** (x **ungleich** 5), **text.equals("Hallo")** (text **ist "Hallo"**),



Getter haben den Zweck, den Wert eines (private) Attributs von anderen Klassen aus zugänglich zu machen. Der Methodenkopf folgt diesem Schema:

```
public TypAttribut getNameAttribut()
```

Setter ermöglichen das Setzen von Attributwerten von außerhalb derselben Klasse. Sie haben einen Parameter mit dem Datentyp des Attributs und Rückgabety `void`. Der Methodenkopf folgt diesem Schema:

Spieler
- int x
- int y
- String imagePath

Besondere Methoden: Konstruktoren



Konstruktoren sind spezielle Methoden, die genutzt werden, um ein **Objekt zu initialisieren/konstruieren**. Ein Konstruktor wird **automatisch beim Erstellen eines Objekts** aufgerufen.

In Java erkennt man einen Konstruktor insb. an zwei Dingen:

1. Er **heißt**
2. Er hat **keinen**

Ansonsten verhält er sich wie eine Methode und kann z.B. Parameter haben, die dann oft als Startwerte für das Objekt dienen.

Ein typischer Konstruktor im Programmcode sieht z.B. so aus:

```
public Spieler(double startX , double startY) {  
    x = startX;  
    y = startY;  
}
```

Im Klassendiagramm würde dieses Beispiel so eingetragen:

Ein neues Objekt erstellt und speichert man dann z.B. so: **Spieler s1 = new Spieler(100,50);**

Besondere Methoden: Konstruktoren



Konstruktoren sind spezielle Methoden, die genutzt werden, um ein **Objekt zu initialisieren/konstruieren**. Ein Konstruktor wird **automatisch beim Erstellen eines Objekts** aufgerufen.

In Java erkennt man einen Konstruktor insb. an zwei Dingen:

1. Er **heißt genauso wie die Klasse** .
2. Er hat **keinen** .

Ansonsten verhält er sich wie eine Methode und kann z.B. Parameter haben, die dann oft als Startwerte für das Objekt dienen.

Ein typischer Konstruktor im Programmcode sieht z.B. so aus:

```
public Spieler(double startX , double startY) {  
    x = startX;  
    y = startY;  
}
```

Im Klassendiagramm würde dieses Beispiel so eingetragen:

Ein neues Objekt erstellt und speichert man dann z.B. so: **Spieler s1 = new Spieler(100,50);**

Besondere Methoden: Konstruktoren



Konstruktoren sind spezielle Methoden, die genutzt werden, um ein **Objekt zu initialisieren/konstruieren**. Ein Konstruktor wird **automatisch beim Erstellen eines Objekts** aufgerufen.

In Java erkennt man einen Konstruktor insb. an zwei Dingen:

1. Er **heißt genauso wie die Klasse** .
2. Er hat **keinen Rückgabety** .

Ansonsten verhält er sich wie eine Methode und kann z.B. Parameter haben, die dann oft als Startwerte für das Objekt dienen.

Ein typischer Konstruktor im Programmcode sieht z.B. so aus:

```
public Spieler(double startX , double startY) {  
    x = startX;  
    y = startY;  
}
```

Im Klassendiagramm würde dieses Beispiel so eingetragen:

Ein neues Objekt erstellt und speichert man dann z.B. so: **Spieler s1 = new Spieler(100,50);**

Besondere Methoden: Konstruktoren



Konstruktoren sind spezielle Methoden, die genutzt werden, um ein **Objekt zu initialisieren/konstruieren**. Ein Konstruktor wird **automatisch beim Erstellen eines Objekts** aufgerufen.

In Java erkennt man einen Konstruktor insb. an zwei Dingen:

1. Er **heißt genauso wie die Klasse** .
2. Er hat **keinen Rückgabetyt** .

Ansonsten verhält er sich wie eine Methode und kann z.B. Parameter haben, die dann oft als Startwerte für das Objekt dienen.

Ein typischer Konstruktor im Programmcode sieht z.B. so aus:

```
public Spieler(double startX , double startY) {  
    x = startX;  
    y = startY;  
}
```

Im Klassendiagramm würde dieses Beispiel so eingetragen:

+ Spieler(double startX, double startY)

Ein neues Objekt erstellt und speichert man dann z.B. so: **Spieler s1 = new Spieler(100,50);**

Besondere Methoden: Main/Hauptprogramm



Möchte man für ein Java-Programm starten, benötigt man eine `Main`-Methode. Diese nennt man auch den **Einstiegspunkt/Hauptprogramm** eines Programms. In ihr werden meistens die benötigten Klassen erstellt und ihre `main`-Methode aufgerufen.

Der **Kopf** der Main-Methode ist immer gleich (siehe Bsp.):

```
public static void main() {  
    Spieler s1 = new Spieler(100,50);  
    // ... weitere Objekte und Methoden  
}
```

Das Schlüsselwort `public static` bedeutet hierbei, dass kein Objekt benötigt wird, um die Methode aufzurufen. In Blockly2Java wird die `main`-Methode automatisch zum Ausführen ausgewählt.

Besondere Methoden: Main/Hauptprogramm



Möchte man für ein Java-Programm starten, benötigt man eine **main-Methode**. Diese nennt man auch den **Einstiegspunkt/Hauptprogramm** eines Programms. In ihr werden meistens die benötigten `Objekte` erstellt und ihre `Methoden` aufgerufen.

Der **Kopf** der Main-Methode ist immer gleich (siehe Bsp.):

```
public static void main() {  
    Spieler s1 = new Spieler(100,50);  
    // ... weitere Objekte und Methoden  
}
```

Das Schlüsselwort `public static` bedeutet hierbei, dass kein Objekt benötigt wird, um die Methode aufzurufen. In Blockly2Java wird die main-Methode automatisch zum Ausführen ausgewählt.

Besondere Methoden: Main/Hauptprogramm



Möchte man für ein Java-Programm starten, benötigt man eine **main-Methode**. Diese nennt man auch den **Einstiegspunkt/Hauptprogramm** eines Programms. In ihr werden meistens die benötigten **Objekt** erstellt und ihre **Methoden** aufgerufen. Der **Kopf** der Main-Methode ist immer gleich (siehe Bsp.):

```
public static void main() {  
    Spieler s1 = new Spieler(100,50);  
    // ... weitere Objekte und Methoden  
}
```

Das Schlüsselwort **public static** bedeutet hierbei, dass kein Objekt benötigt wird, um die Methode aufzurufen. In Blockly2Java wird die main-Methode automatisch zum Ausführen ausgewählt.

Besondere Methoden: Main/Hauptprogramm



Möchte man für ein Java-Programm starten, benötigt man eine **main-Methode**. Diese nennt man auch den **Einstiegspunkt/Hauptprogramm** eines Programms. In ihr werden meistens die benötigten **Objekt** erstellt und ihre **Methoden** aufgerufen. Der **Kopf** der Main-Methode ist immer gleich (siehe Bsp.):

```
public static void main() {  
    Spieler s1 = new Spieler(100,50);  
    // ... weitere Objekte und Methoden  
}
```

Das Schlüsselwort `public static` bedeutet hierbei, dass kein Objekt benötigt wird, um die Methode aufzurufen. In Blockly2Java wird die main-Methode automatisch zum Ausführen ausgewählt.

Besondere Methoden: Main/Hauptprogramm



Möchte man für ein Java-Programm starten, benötigt man eine **main-Methode**. Diese nennt man auch den **Einstiegspunkt/Hauptprogramm** eines Programms. In ihr werden meistens die benötigten **Objekt** erstellt und ihre **Methoden** aufgerufen. Der **Kopf** der Main-Methode ist immer gleich (siehe Bsp.):

```
public static void main() {  
    Spieler s1 = new Spieler(100,50);  
    // ... weitere Objekte und Methoden  
}
```

Das Schlüsselwort **static** bedeutet hierbei, dass kein Objekt benötigt wird, um die Methode aufzurufen. In Blockly2Java wird die main-Methode automatisch zum Ausführen ausgewählt.